COMP 2233: DATA STRUCTURES

Multi-Dimensional Arrays

2D & 3D Matrix Syntax, Memory Allocation, Row-Major Layout, and Applications

Assoc.Prof.Dr. Ömer ÇETİN

Problem Solving Pipeline & Matrix Logic

From Mathematical Problem to Executable Code

- Algorithm Development: Formulating matrix operations and nested loops.
- Program Development: Implementing 2D/3D arrays in C++.
- Program Translation: Compiler verifies array bounds syntax and generates binary.
- Program Execution: Hardware executes linear memory offset calculations.

1. Algorithm Dev.

Design nested loop logic for 2D grids

2. Program Dev.

Code matrix arrays in C++ syntax

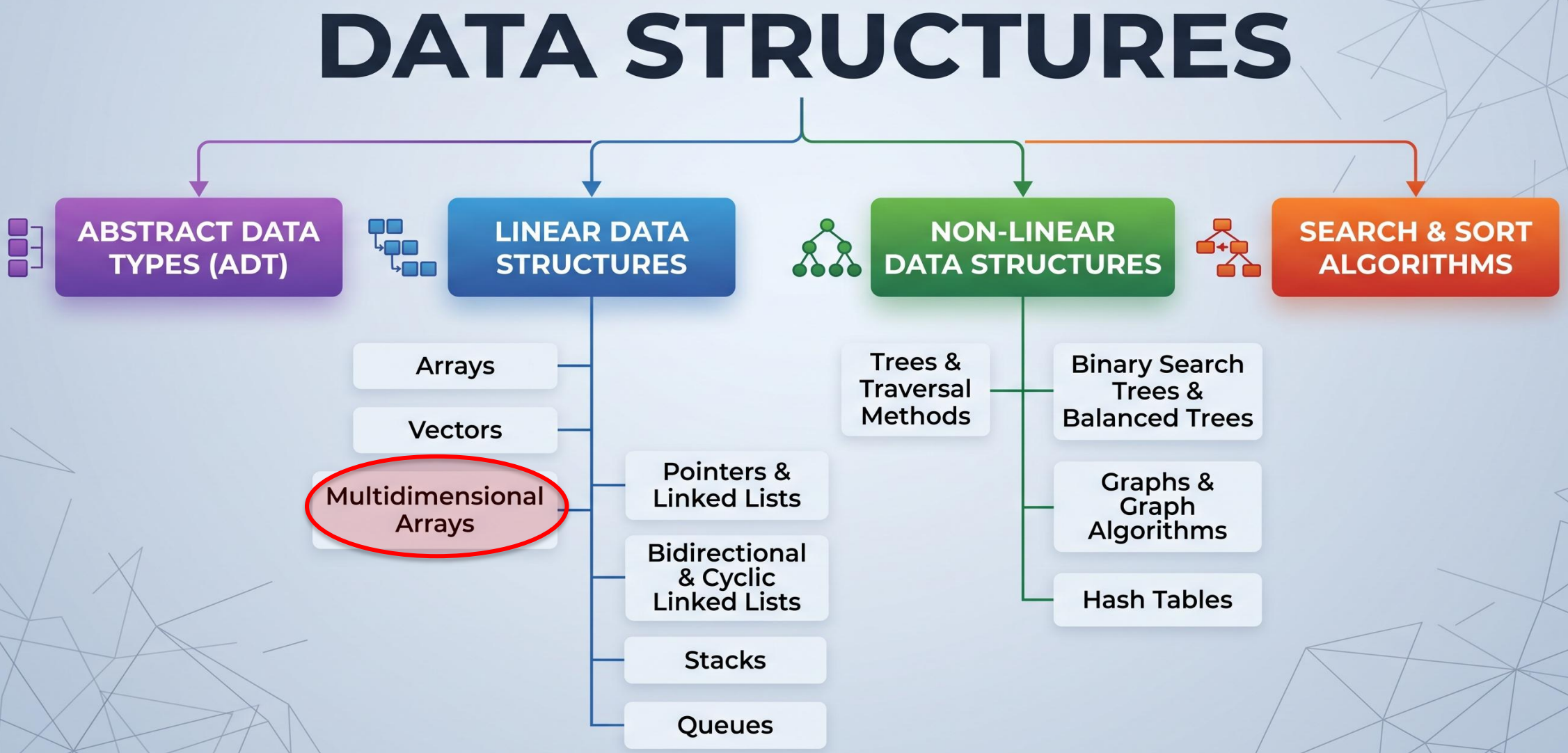
3. Translation

Compiler checks syntax and bounds

4. Execution

CPU computes Row-Major memory addresses

Course Content



What is an Array? Key Characteristics

Fundamental Definition

- **Contiguous Memory Collection:** Elements of the same data type stored in contiguous RAM locations.
- **Random Access O(1):** Directly accessible via index calculations.
- **Fixed Capacity:** Allocated capacity remains static during runtime.
- **Multi-Dimensional Extension:** Arrays containing arrays as their elements.

Key Characteristics

- **Fixed Size:** Declared size cannot change dynamically.
- **Homogeneous Types:** All cells store identical data types (e.g., all `int`).
- **Zero-Based Subindexing:** Indexes start at `0` for both rows and columns.
- **Contiguous Physical Layout:** Multi-dimensional grids are flattened sequentially in RAM.

Array Classification by Dimensions

Dimensional Taxonomy Overview

- **1-Dimensional (1D) Arrays:** Linear list of elements accessed via single index `arr[i]`.
- **2-Dimensional (2D) Arrays:** Grid structure of rows and columns accessed via `arr[row][col]`.
- **3-Dimensional (3D) Arrays:** Cube structure of blocks, rows, and columns accessed via `arr[block][row][col]`.
- **N-Dimensional Arrays:** Higher-order tensors utilized in scientific computing.

1D Vector

Linear Sequence: [A | B | C | D | E] -> **Index: arr[i]**

2D Matrix Grid

Rows x Cols: [A B] / [C D] -> Index: **arr[r][c]**

3D Cube Tensor

Blocks x Rows x Cols: 3D Pixel / Voxel Data -> **Index: arr[b][r][c]**

Syntax of Multi-Dimensional Arrays (2D, 3D, ND)

Declaration Syntax Patterns

- **2D Array Syntax:** `data_type arr[rows][cols];` (e.g., `int arr[3][4];` -> 3 rows, 4 columns).
- **3D Array Syntax:** `data_type arr[blocks][rows][cols];` (e.g., `int arr[2][3][4];`).
- **ND Array Syntax:** Generalizes to N dimensions `arr[d1][d2]...[dN]`.
- **Native Support:** C/C++ natively supports multi-dimensional static array declarations.

```
// Declaration Code Snippets
// 2D Array: 3 Rows, 4 Columns (12 elements
total)
int matrix[3][4];

// 3D Array: 2 Blocks, 3 Rows, 4 Columns (24
elements)
int cube[2][3][4];

// Total Elements Formula:
// Total = Dim1 * Dim2 * ... * DimN
// Total Bytes = Total Elements *
sizeof(data_type)
```

2D Array Creation and Initialization

Initialization Variations

- **Flat Initialization:** `int t[3][4] = {8, 16, 9, 52, 3, 15, 27, 6, 14, 25, 2, 10};`.
- **Nested Row Initialization:** `int t[3][4] = { {8, 16, 9, 52}, {3, 15, 27, 6}, {14, 25, 2, 10} };`.
- **Partial Initialization:** `int t[3][4] = { {8, 16}, {3, 15, 27} };` (unspecified cells set to 0).
- **Readability:** Nested braces `{}` explicitly demarcate row boundaries.

```c
// 2D Initialization Code
#include <stdio.h>

int main(void) {
    // Nested row initialization
    int tablo[3][4] = {
        {8, 16, 9, 52},  // Row 0
        {3, 15, 27, 6},  // Row 1
        {14, 25, 2, 10}  // Row 2
    };

    // Partial initialization (rest become 0)
    int p[3][4] = { {8, 16}, {3, 15, 27} };

    printf("tablo[1][2] = %d\n", tablo[1][2]);
// 27
    printf("p[0][2] = %d\n", p[0][2]);
// 0
    return 0;
}
```

Value Assignment & Row-Major Memory Storage

Memory Mapping & Assignment Syntax

- **Assignment Syntax:** `dizi_adi[row_index][col_index] = value;` (e.g., `Sayilar[5][6] = 67;`).
- **Access Syntax:** `int a = Sayilar[1][5];`.
- **Row-Major Order:** C/C++ stores 2D arrays row by row sequentially in 1D linear RAM.
- **Row-Major Address Formula:** `Address(A[i][j]) = Base + (i * COLS + j) * sizeof(Type)`.

Logical 2D Grid

Row 0: [4, 3, 2, 1] |
Row 1: [1, 2, 3, 4] |
Row 2: [5, 6, 7, 8]

Physical Linear RAM

RAM:
[4, 3, 2, 1 | 1, 2, 3, 4 | 5, 6, 7, 8]

Offset Calculation

Cell (1,2) ->
Offset = 1 * 4 + 2 =
6th RAM Slot

2D Array Traversal using Nested Loops

Nested For Loop Mechanics

- **Outer Loop (`i`):** Iterates through each row index from `0` to `ROWS-1`.
- **Inner Loop (`j`):** Iterates through each column index from `0` to `COLS-1`.
- **Total Operations:** `ROWS * COLS` iterations.
- **Time Complexity:** O(ROWS * COLS) linear time relative to total matrix cells.

```c
// Nested Traversal Code
#include <stdio.h>

int main(void) {
    int arr[3][4] = {
        {8, 16, 9, 52},
        {3, 15, 27, 6},
        {14, 25, 2, 10}
    };

    // Traverse 2D Array
    for (int i = 0; i < 3; i++) {       // Rows
        for (int j = 0; j < 4; j++) {   // Cols
            printf("%d\t", arr[i][j]);
        }
        printf("\n"); // Newline per row
    }
    return 0;
}
```

2D Array Memory Allocation Mapping Diagram

Linearization of 2D Matrix Cells

- **Logical Matrix:** 4x4 Grid `A[0..3, 0..3]` containing 16 character cells.
- **Linear RAM Memory:** Array indexed from 0 to 15 continuously in memory.
- **Mapping Relationship:** `Cell(Row 0, Col 0..3)` maps to RAM slots `0, 1, 2, 3`. `Cell(Row 1, Col 0..3)` maps to RAM slots `4, 5, 6, 7`, and so on.

Logical Row 0

A[0,0]->0,
A[0,1]->1,
A[0,2]->2,
A[0,3]->3

Logical Row 1

A[1,0]->4,
A[1,1]->5,
A[1,2]->6,
A[1,3]->7

Logical Row 3

A[3,0]->12,
A[3,1]->13,
A[3,2]->14,
A[3,3]->15

Practical Applications of Multi-Dimensional Arrays

Core Application Domains

- **Matrix Operations:** Linear algebra, matrix addition, multiplication, determinants.
- **Image Processing:** 2D grayscale pixel grids or 3D RGB color channel buffers.
- **Tables & Databases:** Storing multi-attribute records and spreadsheet data.
- **Game Development:** Board grids and spatial maps (Chess, Tic-Tac-Toe, Sudoku).

Real-World Examples

- **Computer Graphics:** Transformation matrices (translation, rotation, scaling).
- **GIS & Maps:** Altitude grids, terrain elevation maps, heatmaps.
- **Machine Learning:** Tensors in Deep Learning models (e.g., PyTorch/TensorFlow tensors).
- **Scientific Computing:** Weather simulation grids, fluid dynamics meshes.

Example: Calculating Product of Two 3x3 Matrices

3x3 Matrix Multiplication Logic

- **Mathematical Rule:** `CARP[i][j] = Sum(A[i][k] * B[k][j])` for k=0..N-1.
- **Triple Nested Loops:** `i` for rows, `j` for columns, `k` for inner dot product.
- **Time Complexity:** O(N^3) cubic time.
- **Output CARP Matrix:** Product matrix printed row by row.

```
// C++ Matrix Multiplication Code
#include <iostream>
using namespace std;
#define N 3

int main() {
    int A[N][N] = {{1, 0, 0}, {0, 1, 0}, {0, 0, 1}}; //
Identity
    int B[N][N] = {{6, 1, 8}, {7, 2, 5}, {9, 3, 4}};
    int CARP[N][N];

    // Multiply A and B
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            CARP[i][j] = 0;
            for (int k = 0; k < N; k++) {
                CARP[i][j] += A[i][k] * B[k][j];
            }
        }
    }
    // Print CARP matrix...
    return 0;
}
```

Common Mistakes with Multi-Dimensional Arrays

Frequent Pitfalls

- **Indexing Errors:** Accessing `arr[3][4]` when size is declared as `int arr[3][4]` (valid row indexes 0..2, col indexes 0..3).
- **Off-By-One Errors:** Writing loop bounds as `i <= ROWS` instead of `i < ROWS`.
- **Inverted Loop Indexes:** Sweeping `arr[j][i]` instead of `arr[i][j]` causing CPU Cache misses.
- **Uninitialized Cells:** Assuming uninitialized cells are zero without explicit `{0}`.

Memory & Performance Hazards

- **Large Memory Allocation:** Declaring `int huge[1000][1000][1000]` exceeds Stack memory limits!
- **Stack Overflow:** Giant multi-dimensional arrays must be allocated on Heap via dynamic pointers.
- **Cache Inefficiency:** Column-major traversal in Row-Major language causes severe cache misses.

Summary: Multi-Dimensional Arrays

Key Lecture Takeaways

- **Essential Structure:** Multi-dimensional arrays form the backbone of scientific computing and game engineering.
- **Row-Major Memory:** C/C++ flattens 2D/3D grids into linear RAM using direct address arithmetic.
- **Nested Loops:** Traversal and matrix operations require structured nested loop patterns.
- **Next Topics:** Pointers, Memory Address Arithmetic, and Linked Data Structures.

2D Syntax

`type arr[ROWS][COLS]` declarations and row initialization

Row-Major RAM

`Address = Base + (i * COLS + j) * size` direct calculation

Applications

Image processing, games, linear algebra, and matrix multiplication